# Module 11 Notes: Reversing Singly Linked Lists

## 1. Current Progress Analysis

You are ready for **Module 11** because you already know the main ideas needed to understand linked-list reversal.

### From Module 1: Arrays vs Linked Lists

You learned that arrays and linked lists store data differently.

| Structure | Strength | Weakness |
|---|---|---|
| Array | Fast direct access by index | Fixed size and shifting may be needed |
| Linked list | Flexible size and easier insertion/deletion by changing links | No direct jumping to an index |

This matters because Module 11 compares two reversal ideas:

1. Reversing data using an array.
2. Reversing links using pointers.

The array method copies values. The linked-list method changes links.

---

### From Module 2: Node Anatomy

You learned that a singly linked-list node has two parts:

```
[data | next]
```

| Part | Meaning |
|---|---|
| `data` | The value stored in the node |
| `next` | The address of the next node |

C node definition:

```c
struct Node {
    int data;
```

```
    struct Node *next;
};
```

Module 11 mainly works with the `next` part of the node.

---

### From Module 3: Traversal

You learned how to visit every node:

```
p = p->next;
```

Traversal means:

```
Start at first node.
Visit current node.
Move to next node.
Repeat until NULL.
```

All three reversal methods visit the list node by node.

---

### From Module 4: Recursion

You learned that recursion has two phases:

| Phase | Meaning |
|---|---|
| Calling phase | Function calls go deeper |
| Returning phase | Function calls come back |

This matters because recursive reversal moves forward during the calling phase and changes links during the returning phase.

---

### From Module 5: Count Function

You learned how to count nodes:

```
int count(struct Node *p) {
    int c = 0;
```

```
    while (p != NULL) {
        c++;
        p = p->next;
    }

    return c;
}
```

The array reversal method needs the list length, so it uses `count()`.

---

### From Modules 6, 7, 9, and 10: Pointer and Link Changes

You learned how to use pointers such as `p`, `q`, and sometimes `r`.

You also learned that linked-list operations often depend on changing links in the correct order.

For example, insertion used:

```
t->next = p->next;
p->next = t;
```

Deletion used:

```
q->next = p->next;
```

Module 11 continues the same idea, but now we reverse the direction of the entire list.

---

## 2. Big Idea of Module 11

Module 11 teaches how to reverse a singly linked list.

Original list:

```
first
 |
 v
10 -> 20 -> 30 -> 40 -> NULL
```

After reversal:

```
first
 |
 v
40 -> 30 -> 20 -> 10 -> NULL
```

The first node becomes the last node.

The last node becomes the first node.

---

## 3. What Does Reversing a Linked List Mean?

There are two different meanings of reversal.

| Type of reversal | What happens |
|---|---|
| Reverse elements/ data | The nodes stay in the same positions, but their `data` values are copied backward |
| Reverse links | The `data` values stay in their original nodes, but the `next` pointers are reversed |

Example:

```
10 -> 20 -> 30 -> NULL
```

After reversing, we want:

```
30 -> 20 -> 10 -> NULL
```

But there are two ways to get this final visible result.

---

## 4. Reverse Display Is Not Real Reversal

In Module 4, you learned reverse display.

Example:

```c
void RDisplayReverse(struct Node *p) {
    if (p != NULL) {
        RDisplayReverse(p->next);
```

```
        printf("%d ", p->data);
    }
}
```

This prints the list backward.

Original list:

```
10 -> 20 -> 30 -> NULL
```

Output:

```
30 20 10
```

But the actual list is still:

```
10 -> 20 -> 30 -> NULL
```

So reverse display only changes the output order.

It does not change the list.

Module 11 is different because it changes the real list.

---

# 5. The Three Reversal Methods

Module 11 teaches three methods:

1. Array method.
2. Sliding-pointer method.
3. Recursive method.

| Method | Main idea | Extra space |
|---|---|---|
| Array method | Copy data into an array, then copy back in reverse | `O(n)` |
| Sliding-pointer method | Reverse the `next` links using three pointers | `O(1)` |
| Recursive method | Use recursion to reverse links while returning | `O(n)` stack space |

The most important practical method is the **sliding-pointer method**.

---

# 6. Method 1: Reversal Using an Array

## 6.1 Big Idea

The array method reverses the values, not the links.

Steps:

1. Count the number of nodes.
2. Create an array of that size.
3. Copy all node data into the array from left to right.
4. Go back to the first node.
5. Copy array values back into the list from right to left.

---

## 6.2 Example

Original list:

```
10 -> 20 -> 30 -> 40 -> NULL
```

Copy data into array:

```
A = [10, 20, 30, 40]
```

Copy back from the end of the array:

```
40 -> 30 -> 20 -> 10 -> NULL
```

Important:

The nodes did not move.

Only the values inside the nodes changed.

---

## 6.3 Memory Picture of Array Method

Before:

```
Node 1 data = 10
Node 2 data = 20
Node 3 data = 30
Node 4 data = 40
```

Array:

```
A[0] = 10
A[1] = 20
A[2] = 30
A[3] = 40
```

After copying back:

```
Node 1 data = 40
Node 2 data = 30
Node 3 data = 20
Node 4 data = 10
```

The node links are still in the same direction.

---

## 6.4 C Code: Array Reversal

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *first = NULL;

int count(struct Node *p) {
    int c = 0;

    while (p != NULL) {
        c++;
        p = p->next;
    }

    return c;
```

```
}

void ReverseArray(struct Node *p) {
    int n = count(p);

    if (n == 0) {
        return;
    }

    int *A = (int *)malloc(n * sizeof(int));
    int i = 0;

    while (p != NULL) {
        A[i] = p->data;
        p = p->next;
        i++;
    }

    p = first;
    i = n - 1;

    while (p != NULL) {
        p->data = A[i];
        p = p->next;
        i--;
    }

    free(A);
}
```

---

## 6.5 Explanation of Array Reversal Code

### Count the nodes

```
int n = count(p);
```

This finds how many nodes are in the list.

We need this because the array size depends on the number of nodes.

---

### Empty list check

```
if (n == 0) {
    return;
}
```

If the list is empty, there is nothing to reverse.

---

### Create the array

```
int *A = (int *)malloc(n * sizeof(int));
```

This creates an array in heap memory.

Why use `malloc`?

Because the size `n` is known at runtime, not before the program runs.

---

### Copy list values into the array

```
while (p != NULL) {
    A[i] = p->data;
    p = p->next;
    i++;
}
```

This stores every node value in the array.

---

### Reset `p` to the first node

```
p = first;
```

After the first loop, `p` became `NULL`.

So we must move it back to the start.

---

### Copy values back in reverse order

```
i = n - 1;

while (p != NULL) {
    p->data = A[i];
    p = p->next;
    i--;
}
```

This copies the last array value into the first node, then moves backward through the array.

---

### Free the array

```
free(A);
```

Since the array was created using `malloc`, we must release it using `free`.

---

## 6.6 Dry Run: Array Method

Original list:

```
10 -> 20 -> 30 -> 40 -> NULL
```

### Step 1: Copy into array

| Step | `p` points to | Action | Array so far |
|---|---|---|---|
| 1 | 10 | Store 10 | `[10]` |
| 2 | 20 | Store 20 | `[10, 20]` |
| 3 | 30 | Store 30 | `[10, 20, 30]` |
| 4 | 40 | Store 40 | `[10, 20, 30, 40]` |
| 5 | NULL | Stop | `[10, 20, 30, 40]` |

**Step 2: Copy back from the end**

| Node visited | Array index used | Value copied | Node becomes |
|---|---|---|---|
| 1st node | `A[3]` | 40 | 40 |
| 2nd node | `A[2]` | 30 | 30 |
| 3rd node | `A[1]` | 20 | 20 |
| 4th node | `A[0]` | 10 | 10 |

Final list:

```
40 -> 30 -> 20 -> 10 -> NULL
```

---

## 6.7 Advantages of Array Method

It is easy to understand.

It does not require difficult pointer link changes.

It is useful for learning the basic idea of reversal.

---

## 6.8 Disadvantages of Array Method

It uses extra memory.

If the list has `n` nodes, the array also needs `n` spaces.

So extra space is:

```
O(n)
```

It also copies data values.

If each node stores large data, copying can be expensive.

So this method is usually not the best practical method.

---

# 7. Method 2: Reversal Using Sliding Pointers

## 7.1 Big Idea

The sliding-pointer method reverses the actual links.

Original:

```
10 -> 20 -> 30 -> NULL
```

After reversing links:

```
30 -> 20 -> 10 -> NULL
```

Here, the node values stay inside the same nodes.

Only the `next` pointers change direction.

---

## 7.2 Why This Method Is Important

This is the preferred method because:

- It does not need an extra array.
- It reverses the actual links.
- It uses only three pointer variables.
- It uses `O(1)` extra space.

---

## 7.3 The Three Pointers

Sliding reversal uses three pointers:

| Pointer | Simple meaning | Role |
|---|---|---|
| `p` | Current pointer | Points to the node we are processing now |
| `q` | Previous pointer | Points to the node just before `p` |
| `r` | Previous previous pointer | Points to the node before `q` |

At the beginning:

```
p = first;
q = NULL;
r = NULL;
```

Example list:

```
first
 |
 v
10 -> 20 -> 30 -> NULL
```

Initial pointer positions:

```
p -> 10
q -> NULL
r -> NULL
```

---

## 7.4 Core Sliding Sequence

The core sequence is:

```
r = q;
q = p;
p = p->next;
q->next = r;
```

This is the most important part of Module 11.

---

## 7.5 Meaning of Each Line

### Line 1

```
r = q;
```

Move `r` to where `q` was.

---

### Line 2

```
q = p;
```

Move `q` to where `p` was.

---

### Line 3

```
p = p->next;
```

Move `p` forward to the next node.

This must happen before changing `q->next`.

---

### Line 4

```
q->next = r;
```

Reverse the link of the current node.

Instead of pointing forward, `q` now points backward to `r`.

---

## 7.6 Why the Order Matters

The order is very important.

Correct order:

```
r = q;
q = p;
p = p->next;
q->next = r;
```

The key line is:

```
p = p->next;
```

This saves the address of the remaining list before the link is reversed.

If you reverse the link too early, you may lose the rest of the list.

---

## 7.7 Wrong Order Example

Wrong idea:

```
q->next = r;
p = p->next;
```

Why is this dangerous?

Because after changing `q->next`, the original forward link may be gone.

Then `p = p->next` may not move to the correct next node.

Important rule:

```
Move forward first, then reverse the link.
```

---

## 7.8 C Code: Sliding-Pointer Reversal

```c
void ReverseSliding(struct Node *p) {
    struct Node *q = NULL;
    struct Node *r = NULL;

    while (p != NULL) {
        r = q;
        q = p;
        p = p->next;
        q->next = r;
    }

    first = q;
}
```

Call:

```
ReverseSliding(first);
```

---

## 7.9 Explanation of Sliding Code

### Create `q` and `r`

```
struct Node *q = NULL;
struct Node *r = NULL;
```

At the beginning, there is no previous node.

So both `q` and `r` start as `NULL`.

---

### Loop while `p` is valid

```
while (p != NULL)
```

Continue until `p` reaches the end of the original list.

---

### Slide the pointers

```
r = q;
q = p;
p = p->next;
```

This moves the three pointers forward.

---

### Reverse the link

```
q->next = r;
```

This makes the current node point backward.

---

### Update `first`

```
first = q;
```

At the end, `q` points to the old last node.

The old last node is now the new first node.

---

## 7.10 Dry Run: Sliding Reversal

Original list:

```
10 -> 20 -> 30 -> NULL
```

Initial state:

```
p -> 10
q -> NULL
r -> NULL
```

---

### Loop 1

Before:

```
p -> 10 -> 20 -> 30 -> NULL
q -> NULL
r -> NULL
```

Run:

```
r = q;        // r = NULL
q = p;        // q = 10
p = p->next;  // p = 20
q->next = r;  // 10 -> NULL
```

After loop 1:

```
10 -> NULL

p -> 20 -> 30 -> NULL
q -> 10
r -> NULL
```

The old first node now points to `NULL`.

This is correct because after reversal, the old first node becomes the last node.

---

### Loop 2

Run:

```
r = q;        // r = 10
q = p;        // q = 20
p = p->next;  // p = 30
q->next = r;  // 20 -> 10
```

After loop 2:

```
20 -> 10 -> NULL

p -> 30 -> NULL
q -> 20
r -> 10
```

---

### Loop 3

Run:

```
r = q;        // r = 20
q = p;        // q = 30
p = p->next;  // p = NULL
q->next = r;  // 30 -> 20
```

After loop 3:

```
30 -> 20 -> 10 -> NULL

p -> NULL
q -> 30
r -> 20
```

Now `p == NULL`, so the loop stops.

Finally:

```
first = q;
```

Final list:

```
first
 |
 v
30 -> 20 -> 10 -> NULL
```

---

## 7.11 Why `first = q` Is Needed

At the end:

```
p == NULL
q points to the old last node
```

The old last node has become the new first node.

So we must write:

```
first = q;
```

If we forget this, `first` may still point to the old first node.

But the old first node is now the last node.

That would make the list look broken from the program's point of view.

---

## 7.12 Sliding Method on a One-Node List

Original:

```
10 -> NULL
```

Initial:

```
p -> 10
q -> NULL
r -> NULL
```

After one loop:

```
10 -> NULL
p -> NULL
q -> 10
```

Then:

```
first = q;
```

Final:

```
10 -> NULL
```

A one-node list stays the same.

---

## 7.13 Sliding Method on an Empty List

Original:

```
first = NULL
```

Call:

```
ReverseSliding(first);
```

Inside the function:

```
p == NULL
```

The loop does not run.

Then:

```
first = q;
```

Since `q` is `NULL`, `first` stays `NULL`.

So an empty list stays empty.

---

# 8. Method 3: Recursive Reversal

## 8.1 Big Idea

Recursive reversal reverses the links using function calls.

It is similar to sliding-pointer reversal, but instead of a loop, we use recursion.

The function receives two pointers:

| Pointer | Meaning |
|---|---|
| `q` | Previous node |
| `p` | Current node |

Initial call:

```
ReverseRecursive(NULL, first);
```

---

## 8.2 Recursive Reversal Code

```
void ReverseRecursive(struct Node *q, struct Node *p) {
    if (p != NULL) {
        ReverseRecursive(p, p->next);
        p->next = q;
    } else {
        first = q;
    }
}
```

Call:

```
ReverseRecursive(NULL, first);
```

---

## 8.3 How Recursive Reversal Works

Original list:

```
10 -> 20 -> 30 -> NULL
```

Initial call:

```
ReverseRecursive(NULL, 10)
```

This means:

```
Previous node = NULL
Current node = 10
```

Then the function calls itself with:

```
ReverseRecursive(p, p->next);
```

So the calls move forward through the list.

---

## 8.4 Calling Phase

For the list:

```
10 -> 20 -> 30 -> NULL
```

The recursive calls are:

```
ReverseRecursive(NULL, 10)
ReverseRecursive(10, 20)
ReverseRecursive(20, 30)
ReverseRecursive(30, NULL)
```

At the last call:

```
p == NULL
q == 30
```

So the function does:

```
first = q;
```

Now `first` points to 30.

---

## 8.5 Returning Phase

After reaching `NULL`, the function calls begin to return.

During returning, this line runs:

```
p->next = q;
```

This reverses each link.

Returning steps:

| Returning from node | Link changed |
|---|---|
| 30 | `30->next = 20` |

| Returning from node | Link changed |
|---|---|
| 20 | `20->next = 10` |
| 10 | `10->next = NULL` |

Final list:

```
30 -> 20 -> 10 -> NULL
```

---

## 8.6 Dry Run: Recursive Reversal

Original list:

```
10 -> 20 -> 30 -> NULL
```

Call:

```
ReverseRecursive(NULL, first);
```

### Calling phase

```
ReverseRecursive(NULL, 10)
ReverseRecursive(10, 20)
ReverseRecursive(20, 30)
ReverseRecursive(30, NULL)
```

At `p == NULL`:

```
first = q;
```

So:

```
first -> 30
```

### Returning phase

Step 1:

```
30->next = 20;
```

Step 2:

```
20->next = 10;
```

Step 3:

```
10->next = NULL;
```

Final result:

```
30 -> 20 -> 10 -> NULL
```

---

## 8.7 Why Recursive Reversal Uses Extra Space

Recursive reversal uses the system stack.

For each node, one function call is stored on the stack.

If the list has `n` nodes, recursive reversal uses about `n` stack frames.

So extra space is:

```
O(n)
```

This is why recursive reversal is elegant, but usually less preferred than the sliding-pointer method.

---

# 9. Full C Program for Module 11

This program includes:

- Node definition.
- Create function.
- Display function.
- Count function.
- Array reversal.

- Sliding-pointer reversal.
- Recursive reversal.

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *first = NULL;

void create(int A[], int n) {
    if (n <= 0) {
        first = NULL;
        return;
    }

    struct Node *t;
    struct Node *last;

    first = (struct Node *)malloc(sizeof(struct Node));
    first->data = A[0];
    first->next = NULL;
    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;
        last->next = t;
        last = t;
    }
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}

int count(struct Node *p) {
    int c = 0;

    while (p != NULL) {
```

```
        c++;
        p = p->next;
    }

    return c;
}

void ReverseArray(struct Node *p) {
    int n = count(p);

    if (n == 0) {
        return;
    }

    int *A = (int *)malloc(n * sizeof(int));
    int i = 0;

    while (p != NULL) {
        A[i] = p->data;
        p = p->next;
        i++;
    }

    p = first;
    i = n - 1;

    while (p != NULL) {
        p->data = A[i];
        p = p->next;
        i--;
    }

    free(A);
}

void ReverseSliding(struct Node *p) {
    struct Node *q = NULL;
    struct Node *r = NULL;

    while (p != NULL) {
        r = q;
        q = p;
        p = p->next;
        q->next = r;
    }

    first = q;
}
```

```
void ReverseRecursive(struct Node *q, struct Node *p) {
    if (p != NULL) {
        ReverseRecursive(p, p->next);
        p->next = q;
    } else {
        first = q;
    }
}

int main() {
    int A[] = {10, 20, 30, 40, 50};

    create(A, 5);

    printf("Original list: ");
    display(first);

    ReverseSliding(first);
    printf("\nAfter sliding reverse: ");
    display(first);

    ReverseRecursive(NULL, first);
    printf("\nAfter recursive reverse: ");
    display(first);

    ReverseArray(first);
    printf("\nAfter array reverse: ");
    display(first);

    return 0;
}
```

Possible output:

```
Original list: 10 20 30 40 50
After sliding reverse: 50 40 30 20 10
After recursive reverse: 10 20 30 40 50
After array reverse: 50 40 30 20 10
```

Important:

Every reversal changes the current list.

So reversing twice gives the original order again.

---

# 10. Complexity Analysis

Let `n` be the number of nodes.

| Method | Time complexity | Extra space | Why |
|---|---|---|---|
| Array reversal | `O(n)` | `O(n)` | Visits all nodes and uses an extra array |
| Sliding-pointer reversal | `O(n)` | `O(1)` | Visits all nodes and uses only three pointers |
| Recursive reversal | `O(n)` | `O(n)` | Visits all nodes and uses recursive stack frames |

---

## 10.1 Why All Methods Are `O(n)` Time

Every method must visit every node at least once.

So the time depends on the number of nodes.

If there are 5 nodes, we process 5 nodes.

If there are 500 nodes, we process 500 nodes.

So time complexity is:

```
O(n)
```

---

## 10.2 Why Sliding Reversal Is `O(1)` Extra Space

Sliding reversal only uses:

```c
struct Node *p;
struct Node *q;
struct Node *r;
```

That is only a fixed number of pointers.

It does not matter whether the list has 5 nodes or 5000 nodes.

So extra space is:

```
O(1)
```

---

### 10.3 Why Recursive Reversal Is `O(n)` Extra Space

Recursive reversal stores one function call for each node.

For this list:

```
10 -> 20 -> 30 -> NULL
```

The calls are:

```
ReverseRecursive(NULL, 10)
ReverseRecursive(10, 20)
ReverseRecursive(20, 30)
ReverseRecursive(30, NULL)
```

The system stack stores these calls.

For `n` nodes, this uses `O(n)` stack space.

---

## 11. Comparing the Three Methods

| Feature | Array method | Sliding-pointer method | Recursive method |
|---|---|---|---|
| Changes node data? | Yes | No | No |
| Changes links? | No | Yes | Yes |
| Uses extra array? | Yes | No | No |
| Uses recursion stack? | No | No | Yes |
| Extra space | `O(n)` | `O(1)` | `O(n)` |
| Best practical method? | No | Yes | Usually no |
| Beginner difficulty | Easy | Medium | Medium/Hard |

---

### 11.1 Which Method Should You Prefer?

The best practical method is:

```
Sliding-pointer reversal
```

Why?

Because it reverses the actual links and uses only `O(1)` extra space.

---

# 12. Common Beginner Mistakes

## Mistake 1: Thinking Reverse Display Reverses the List

Reverse display only prints backward.

It does not change the actual links.

---

## Mistake 2: Forgetting to Move `p` Before Changing `q->next`

Wrong idea:

```
q->next = r;
p = p->next;
```

Correct idea:

```
p = p->next;
q->next = r;
```

Reason:

You must save the next node before breaking the old link.

---

## Mistake 3: Forgetting `first = q`

At the end of sliding reversal:

```
q points to the new first node
```

So you must write:

```
first = q;
```

---

## Mistake 4: Thinking the Array Method Reverses Links

The array method does not reverse links.

It only copies data values in reverse order.

---

## Mistake 5: Forgetting to Free the Array

If you use:

```
malloc
```

then you should later use:

```
free
```

So the array method should end with:

```
free(A);
```

---

## Mistake 6: Not Handling an Empty List

An empty list is:

```
first = NULL
```

Good reversal code should not crash when the list is empty.

The sliding method naturally handles this because the loop does not run if `p == NULL`.

# 13. Important Questions and Answers

## Question 1: Why do we need `malloc` in the array method?

Because the number of nodes is known only when the program runs.

So we create the array dynamically:

```
int *A = (int *)malloc(n * sizeof(int));
```

---

## Question 2: What does `q->next = r` do?

It reverses the link of the current node.

Before:

```
q -> next node
```

After:

```
q -> previous node
```

So the link direction changes.

---

## Question 3: Why is `p = p->next` done before `q->next = r`?

Because we need to remember the next node before changing the current node's link.

If we change the link first, we may lose access to the rest of the list.

---

## Question 4: When are recursive links reversed?

They are reversed during the returning phase.

The function first goes all the way to `NULL`.

Then, while returning, it runs:

```
p->next = q;
```

---

### Question 5: Why is the sliding-pointer method preferred?

Because it:

- Reverses the actual links.
- Does not copy data.
- Does not use an extra array.
- Does not use recursion stack space.
- Uses only `O(1)` extra space.

---

# 14. Final Summary

Module 11 teaches how to reverse a singly linked list.

There are three methods:

## 1. Array Method

Copy node values into an array, then copy them back in reverse order.

```
Extra space: O(n)
```

Easy to understand, but not the best method.

---

## 2. Sliding-Pointer Method

Reverse the actual `next` links using three pointers:

```
p, q, r
```

Core code:

```
r = q;
q = p;
p = p->next;
q->next = r;
```

Final step:

```
first = q;
```

This is the preferred method.

```
Extra space: O(1)
```

---

### 3. Recursive Method

Use recursion to go to the end of the list, then reverse links while returning.

Important line:

```
p->next = q;
```

Elegant, but uses stack space.

```
Extra space: O(n)
```

---

# 15. The Most Important Thing to Remember

The most important code pattern in Module 11 is:

```
r = q;
q = p;
p = p->next;
q->next = r;
```

And the most important final step is:

```
first = q;
```

If you understand why these lines work, you understand the core idea of reversing a singly linked list.

---

# 16. Quick Revision Checklist

Before moving to Module 12, make sure you can answer these:

1. What is the difference between reverse display and real reversal?
2. What does the array method reverse: data or links?
3. Why does the array method use `O(n)` extra space?
4. What are the roles of `p`, `q`, and `r`?
5. Why must `p = p->next` happen before `q->next = r`?
6. Why do we write `first = q` at the end?
7. During which phase does recursive reversal change links?
8. Which reversal method is preferred and why?

If you can explain these clearly, you have understood Module 11 well.